

ASE Introduction

October 29, 2025

1 The Atomic Simulation Environment - An Introduction

1.1 What is ASE?

ASE is Python library for the atomistic simulation of molecules and materials.
<https://wiki.fysik.dtu.dk/ase/index.html>

Atomic Simulation Environment

The Atomic Simulation Environment (ASE) is a set of tools and Python modules for setting up, manipulating, running, visualizing and analyzing atomistic simulations. The code is freely available under the [GNU LGPL license](#).

ASE provides interfaces to different codes through `Calculators` which are used together with the central `Atoms` object and the many available algorithms in ASE.

```
>>> # Example: structure optimization of hydrogen molecule
>>> from ase import Atoms
>>> from ase.optimize import BFGS
>>> from ase.calculators.nwchem import NWChem
>>> from ase.io import write
>>> h2 = Atoms('H2',
...           positions=[[0, 0, 0],
...                     [0, 0, 0.7]])
>>> h2.calc = NWChem(xc='PBE')
>>> opt = BFGS(h2)
>>> opt.run(fmax=0.02)
BFGS: 0 19:10:49 -31.435229 2.2691
BFGS: 1 19:10:50 -31.490773 0.3740
BFGS: 2 19:10:50 -31.492791 0.0630
BFGS: 3 19:10:51 -31.492848 0.0023
>>> write('H2.xyz', h2)
>>> h2.get_potential_energy()
-31.492847808329216
```

Supported Calculators

ABACUS Abinit asap Atomistica BigDFT CASTEP CP2K CRYSTAL17 DeePMD-kit deMon DFT-D4 DFTK exciting

ASE

Search docs

About

Installation

- Requirements
 - Installation using system package managers
 - Installation from PyPI using pip
 - Installation from source
 - Test your installation
- Getting started
- Tutorials
- Modules
- Command line tool
- Tips and tricks
- Gallery
- Release notes
- Contact
- ASE ecosystem
- Development
- Frequently Asked Questions
- ASE Workshop 2019

Installation

Requirements

- Python 3.9 or newer
- NumPy (base N-dimensional array package)
- SciPy (library for scientific computing)
- Matplotlib (plotting)

Optional:

- Flask for ase.db web-interface
- pytest for running tests
- pytest-mock for running some more tests
- pytest-xdist for running tests in parallel
- spglib for certain symmetry-related features

Installation using system package managers

Linux

Major GNU/Linux distributions (including Debian and Ubuntu derivatives, Arch, Fedora, Red Hat and CentOS) have a `python-ase` package available that you can install on your system. This will manage dependencies and make ASE available for all users.

Note

Depending on the distribution, this may not be the latest release of ASE.

Max OSX (Homebrew)

It is generally not recommended to rely on the Mac system Python. Mac OSX versions before 12.3 included an old Python, incompatible with ASE and missing the `pip` package manager. Since 12.3 MacOS has moved to newer Python versions but it may not be installed by default. Over time, different approaches to

It contains modules for working with atoms

ASE

Search docs

About

Installation

Getting started

Tutorials

Modules

The Atoms object

The Cell object

Units

File input and output

Building things

Equation of state

Chemical formula type

Chemical symbols

Collections

The data module

Structure optimization

Molecular dynamics

Constraints

Filters

Using the spacegroup subpackage

Building neighbor-lists

Geometry tools

A database for atoms

Minimum energy path

index | modules | [gitlab](#) | [page source](#)

Modules

- `ase (Atom)`
- `ase (Atoms)`
- `build`
- `cell`
- `calculators`
- `collections`
- `constraints`
- `db`
- `dft`

- `data`
- `filters`
- `formula`
- `ga`
- `geometry`
- `gui`
- `io`
- `lattice`

- `md`
- `mep`
- `neighborlist`
- `optimize`
- `parallel`
- `phasediagram`
- `phonons`
- `spacegroup`

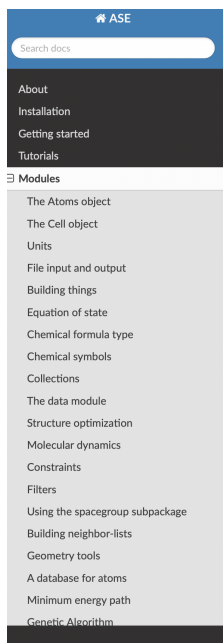
- `spectrum`
- `symbols`
- `transport`
- `thermochemistry`
- `units`
- `utils`
- `vibrations`
- `visualize`

See also

- [Tutorials](#)
- [Command line tool](#)
- [Source code](#)
- Presentation about ASE: [ase-talk.pdf](#)

- [The Atoms object](#)
 - Working with the array methods of Atoms objects
 - Unit cell and boundary conditions
 - Special attributes
 - Adding a calculator
 - List-methods
 - Other methods
 - List of all Methods
- [The Cell object](#)
 - `Cell`

Most importantly is the calculator module



Calculators

[index](#) | [modules](#) | [gitlab](#) | [page source](#)

For ASE, a calculator is a black box that can take atomic numbers and atomic positions from an `Atoms` object and calculate the energy and forces and sometimes also stresses.

In order to calculate forces and energies, you need to attach a calculator object to your atoms object:

```

>>> atoms = read('molecule.xyz')
>>> e = atoms.get_potential_energy()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    File "/home/jjmo/ase/atoms/ase.py", line 399, in get_potential_energy
      raise RuntimeError('Atoms object has no calculator.')
RuntimeError: Atoms object has no calculator.
>>> from ase.calculators.abinit import Abinit
>>> calc = Abinit(...)
>>> atoms.calc = calc
>>> e = atoms.get_potential_energy()
>>> print(e)
-42.0

```

Here we attached an instance of the `ase.calculators.abinit` class and then we asked for the energy.

Supported calculators

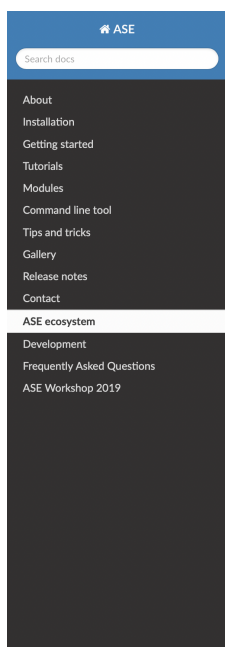
The calculators can be divided in four groups:

1. Abacus, ALIGNN, AMS, Asap, BigDFT, CHGNet, DeePMD-kit, DFTD3, DFTD4, DFTK, FLEUR, GPAW, Hotbit, M3GNet, MACE, TBLite, and XTB have their own native or external ASE interfaces.
2. ABINIT, AMBER, CP2K, CASTEP, deMon2k, DFTB+, ELK, EXCITING, FHI-aims, GAUSSIAN, Gromacs, LAMMPS, MOPAC, NWChem, Octopus, ONETEP, PLUMED, psi4, Q-Chem, Quantum ESPRESSO, SIESTA, TURBOMOLE and VASP, have Python wrappers in the ASE package, but the actual FORTRAN/C/C++ codes are not part of ASE.
3. Pure python implementations included in the ASE package: EMT, EAM, Lennard-Jones, Morse and HarmonicCalculator.
4. Calculators that wrap others, included in the ASE package:
 - `ase.calculators.checkpoint.CheckpointCalculator`
 - `ase.calculators.fd.FiniteDifferenceCalculator`

This means that ASE is very flexible and can interface with different codes and ultimately different levels of theory



Since ASE is written in Python, it also interfaces with other Python codes for materials modelling, which creates an ecosystem of software and enhances our scientific toolbox.

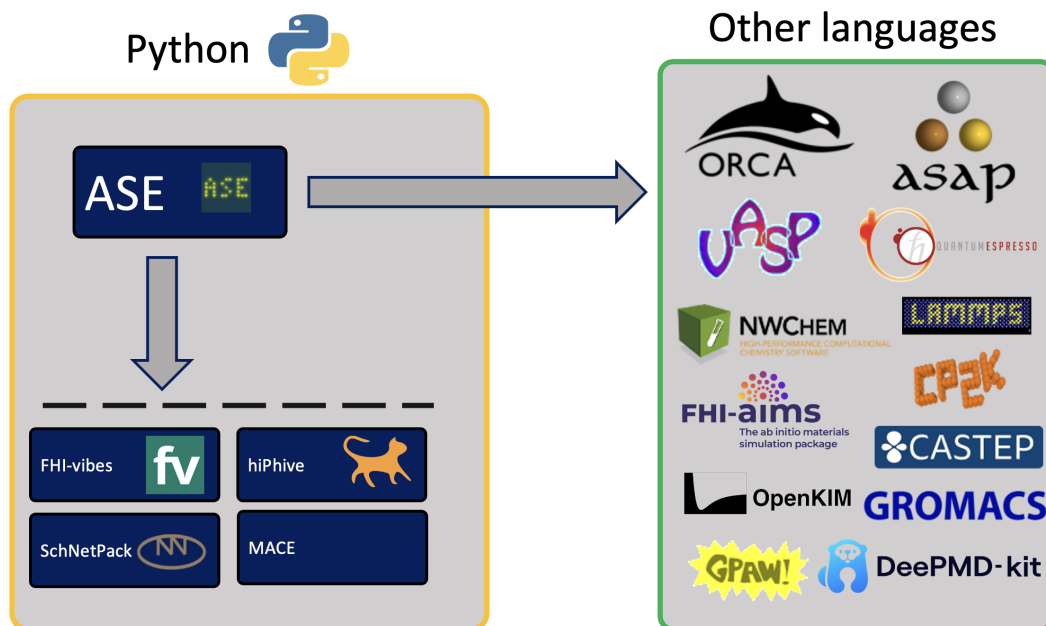


ASE ecosystem

This is a list of software packages related to ASE or using ASE. These could well be of interest to ASE users in general. If you know of a project which should be listed here, but isn't, please open a merge request adding link and descriptive paragraph.

Listed in alphabetical order, for want of a better approach.

- **abTEM**: abTEM provides a Python API for running simulations of (scanning) transmission electron microscopy images and diffraction patterns.
- **ACAT**: ACAT is a Python package for atomistic modelling of metal or alloy heterogeneous catalysts. ACAT provides automatic identification of adsorption sites and adsorbate coverages for a wide range of surfaces and nanoparticles. ACAT also provides tools for structure generation and global optimization of catalysts with and without adsorbates.
- **AGOX**: The Atomistic Global Optimization X package contains a collection of tools for global optimization of atomic systems. The package allows running a variety of standard global optimization algorithms, such as random structure search, basin-hopping in addition to machine-learning enhanced algorithms like GOFEE. Any ASE calculator can be used as the objective function for the optimization.
- **atomicrex**: atomicrex is a versatile tool for the construction of interatomic potential models. It includes a Python interface for integration with first-principles codes via ASE as well as other Python libraries.
- **CHGNet**: A pretrained universal neural network potential for charge-informed atomistic modeling
- **CLEASE**: CCluster Expansion in Atomic Simulation Environment (CLEASE) is a package that automates the cumbersome setup and construction procedure of cluster expansion (CE). It provides a comprehensive list of tools for specifying parameters for CE, generating training structures, fitting effective cluster interaction (ECI) values and running Monte Carlo simulations.
- **COGEF**: COnstrained Geometries simulate External Force. This package is useful for analysing properties of bond-breaking reactions, such as how much force is required to break a chemical bond.
- **DebyeCalculator**: A vectorised implementation of the Debye Scattering Equation on CPU and GPU to calculate the scattering intensity $I(Q)$, the Total Scattering Structure Function $S(Q)$, the Reduced Total Scattering Function $F(Q)$, or the Reduced Atomic Pair Distribution Function $G(r)$ from an atomic structure. Use DebyeCalculator to simulate powder diffraction, total scattering with pair distribution function or small-angle scattering data of finite systems such as nanoparticles.
- **effmass**: Calculates various definitions of effective mass from the electronic bandstructure of a semiconductor.
- **evgraf**: A python library for crystal reduction (i.e. finding primitive cells), and identification and symmetrization of structures with inversion pseudosymmetry.
- **FHI-vibes**: A python package for calculating and analyzing the vibrational properties of solids from first principles. FHI-vibes bridges between the harmonic approximation and fully anharmonic molecular dynamics simulations. FHI-vibes builds on several existing packages including ASE, and provides a consistent and user-friendly interface.
- **gpatom**: A Python package which provides several tools for geometry optimisation and related tasks in atomistic systems using machine learning surrogate models. gpatom is an extension to the Atomic Simulation Environment.



We can build all sorts of structures with ASE from molecules to crystals

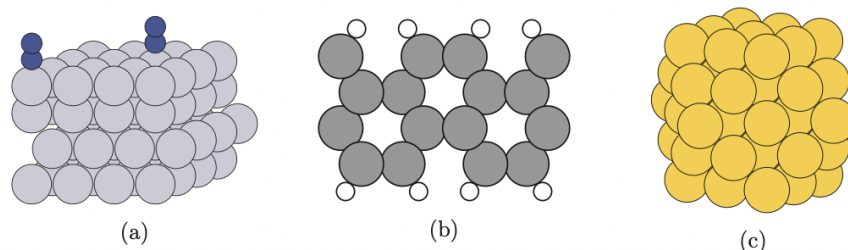
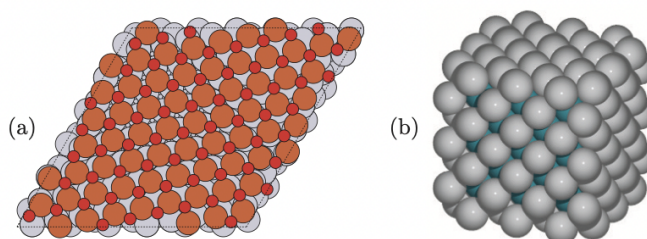


Figure 2. (a) Platinum surface with 2 N₂ adsorbed at top sites [55], (b) carbon nanoribbon with H-saturated edge, and (c) cuboctahedral gold nanoparticle constructed using various functions of ASE.



1.2 OK but how do we actually use ASE to model materials?

We can do a lot of things with ASE. We can use it to:

- Build materials
- Simulate materials (Vibrations, energy calculations, molecular dynamics, transition states)
- Visualise simulations

Let's take a look at an example calculation

```
[2]: from ase import Atoms
from ase.calculators.emt import EMT
from ase.visualize import view

atom = Atoms('N')
atom.calc = EMT()
e_atom = atom.get_potential_energy()

d = 1.1
molecule = Atoms('2N', [(0., 0., 0.), (0., 0., d)])
molecule.calc = EMT()
e_molecule = molecule.get_potential_energy()

e_atomization = e_molecule - 2 * e_atom

print('Nitrogen atom energy: %5.2f eV' % e_atom)
print('Nitrogen molecule energy: %5.2f eV' % e_molecule)
print('Atomization energy: %5.2f eV' % -e_atomization)
```

```
view(molecule, viewer="ngl")
```

Nitrogen atom energy: 5.10 eV
Nitrogen molecule energy: 0.44 eV
Atomization energy: 9.76 eV

```
HBox(children=(NGLWidget(), VBox(children=(Dropdown(description='Show',  
↳options=('All', 'N'), value='All'), Dr...
```

```
[4]: from ase.build import nanotube  
cnt1 = nanotube(6, 0, length=4)  
cnt2 = nanotube(3, 3, length=6, bond=1.4, symbol='Si')  
view(cnt1, viewer="ngl")
```

```
HBox(children=(NGLWidget(), VBox(children=(Dropdown(description='Show',  
↳options=('All', 'Si'), value='All'), D...
```

```
[6]: import numpy as np  
  
from ase import Atoms  
from ase.build import molecule  
from ase.io import write  
  
a = 5.64 # Lattice constant for NaCl  
cell = [a / np.sqrt(2), a / np.sqrt(2), a]  
atoms = Atoms(symbols='Na2Cl2', pbc=True, cell=cell,  
              scaled_positions=[(.0, .0, .0),  
                                (.5, .5, .5),  
                                (.5, .5, .0),  
                                (.0, .0, .5)]) * (3, 4, 2) + molecule('C6H6')  
  
# Move molecule to 3.5Ang from surface, and translate one unit cell in xy  
atoms.positions[-12:, 2] += atoms.positions[:-12, 2].max() + 3.5  
atoms.positions[-12:, :2] += cell[:2]  
  
# Mark a single unit cell  
atoms.cell = cell  
  
view(atoms, viewer="ngl")
```

```
HBox(children=(NGLWidget(), VBox(children=(Dropdown(description='Show',  
↳options=('All', 'Cl', 'H', 'Na', 'C'),...
```

Once we've created them we can start playing around with them. We have the method and now we can create anything from any ingredients

